

Microfrontales II

Cómo compartir código entre microfrontales en tiempo de ejecución con module federation

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Miguel Ángel Muñoz

Contenido

1. Qué es la federación de módulos.....	3
2. Ventajas de la federación de módulos (y cuándo evitarla).....	3
3. Host vs. remote: los dos papeles de un MFE	4
4. Configurando module federation	4
4.1. Los parámetros clave del plugin.....	4
4.2. Analizando un ejemplo completo	5
5. Dependencias compartidas.....	5
6. Cómo funciona module federation por dentro.....	6
7. Qué hemos aprendido.....	7

1. Qué es la federación de módulos

En el documento anterior vimos qué son los microfrontales y cómo Domain-Driven Design ayuda a definir sus límites. Toca ahora ver cómo sacarles el máximo partido: compartiendo código entre ellos de una forma mucho más potente que un simple paquete de NPM. A eso responde module federation.

Module federation es una funcionalidad introducida por Webpack a partir de su versión 5. En esencia, resuelve un problema muy concreto: cómo compartir código entre aplicaciones —microfrontales— de forma sencilla, sin duplicarlo y sin tener que recompilar cada aplicación consumidora cada vez que ese código cambia.

Cuatro ideas resumen su funcionamiento:

- ▶ Permite compartir código entre aplicaciones de forma muy sencilla.
- ▶ Se encarga de exponer las dependencias en tiempo de ejecución, no en tiempo de compilación.
- ▶ También permite exponer el propio código de una aplicación como una dependencia remota que otras pueden consumir.
- ▶ Exige dos requisitos: trabajar con módulos y usar Webpack 5.

Un ejemplo ayuda a entender el problema que viene a resolver. Imaginemos dos aplicaciones de un banco: Posición global, la home donde se ve el estado de cuentas, tarjetas y préstamos, y Cuentas, donde se detallan las cuentas del cliente y sus movimientos. En Posición global existe un componente carrusel que muestra las cuentas del cliente, alimentado por una llamada a una API. Ahora queremos reutilizar ese mismo carrusel en Cuentas.

Una primera opción sería publicar el componente en una librería de NPM y que ambas aplicaciones la instalen. El problema es que solo se reutiliza el "cascarón" visual: la lógica de negocio y las llamadas a la API se acaban duplicando en cada aplicación que lo instale.

Una segunda opción sería crear un MFE completo —con toda su lógica de negocio incluida— y publicarlo también vía NPM, para que se construya en tiempo de compilación. Aquí surge un problema distinto: cada vez que hay que modificar ese MFE, hay que actualizar su versión en todas las aplicaciones que lo usan y volver a desplegarlas todas. Además, si no se actualizan todas a la vez, conviven versiones distintas del mismo MFE.

La solución es module federation: se crea el MFE de cuentas, se expone su código como un módulo federado, y el resto de MFEs que lo necesiten lo cargan directamente en tiempo de ejecución. El resultado es que cualquier cambio en el MFE de cuentas se actualiza automáticamente en todos los MFEs dependientes, sin que nadie tenga que volver a desplegar nada.

★ Lo más importante

- ▶ Module federation resuelve lo que un simple paquete de NPM no puede: mantener sincronizada, en tiempo de ejecución, una única fuente de verdad del código compartido.
- ▶ Solo exige dos cosas: trabajar con módulos y usar Webpack 5.
- ▶ El equipo dueño del MFE compartido puede modificarlo sin que el resto de equipos tenga que hacer nada: se actualiza automáticamente en cuanto se despliega.

2. Ventajas de la federación de módulos (y cuándo evitarla)

Antes de entrar en la configuración conviene tener claro qué garantías ofrece module federation frente a otras soluciones:

- ▶ El código permanece en su lugar: pertenece a una única aplicación, se renderiza tal cual y no se modifica por muchas aplicaciones que lo consuman.
- ▶ No exige ningún framework propio: basta con que las aplicaciones que comparten dependencias usen el mismo framework. Si necesitas hacer convivir aplicaciones de frameworks distintos (React, Angular, Vue...), se puede combinar con SingleSPA, siempre que se prescinda de SystemJS.
- ▶ No necesita cargadores de código adicionales: otros frameworks de microfrontales, como Single SPA, se apoyan en cargadores como SystemJS que trabajan junto a los imports de Babel y Webpack. Los módulos federados funcionan como cualquier módulo importado normal, sin cargador extra.
- ▶ Aplica a cualquier código JavaScript: mientras que los frameworks de microfrontales suelen limitarse a componentes visuales, module federation puede compartir cualquier cosa —componentes, lógica de negocio, cadenas de i18n, lo que sea código JavaScript.
- ▶ Es universal: funciona en cualquier plataforma que ejecute JavaScript en tiempo de ejecución — navegador, Node, Electron, Web Worker— y es compatible con formatos como UMD, AMD, CommonJS o SystemJS.

Dos avisos importantes antes de lanzarse a usarla:

- ▶ Es una técnica compleja, pensada para desarrolladores con buen dominio del ecosistema. Eso no significa evitarla, sino empezar a interiorizarla cuanto antes.
- ▶ Solo tiene sentido si necesitas cargar dependencias en tiempo de ejecución. Si tu plan es cargar todo en tiempo de compilación, complicarte con module federation no aporta nada.

★ Regla clave

- ▶ Antes de incorporar module federation a un proyecto, pregúntate si realmente necesitas actualizar código compartido en tiempo de ejecución sin recompilar. Si la respuesta es no, un paquete de NPM tradicional es más sencillo y sigue siendo perfectamente válido.

3. Host vs. remote: los dos papeles de un MFE

Antes de tocar configuración hace falta fijar dos conceptos que aparecen constantemente al hablar de module federation: host y remote — los mismos roles que ya vimos, en otro contexto, al hablar del Application Shell.

- ▶ MFE host: es el que hospeda y consume los módulos federados de otras aplicaciones.
- ▶ MFE remote: es el que expone su propio código —a sí mismo o módulos concretos— para que otras aplicaciones lo consuman.

Un mismo MFE puede ejercer ambos papeles a la vez: actúa como host cuando consume módulos expuestos por otro MFE, y como remote cuando expone su propio código para que otros lo usen. No son roles fijos ni excluyentes, sino dos funciones que una misma aplicación puede combinar según lo que necesite en cada momento.

Para no perder tiempo montando la infraestructura desde cero mientras se aprende, existe una herramienta de NPM que crea aplicaciones con module federation ya preconfigurado: create-mf-app. Basta con ejecutar `npx create-mf-app` en la terminal para arrancar un proyecto de este tipo.

- ▶ `npx` es una utilidad que viene con NPM y permite ejecutar directamente paquetes alojados en su repositorio, sin tener que instalarlos primero de forma global.
- ▶ `yarn` es una alternativa a NPM: instala dependencias y ejecuta scripts de la misma manera, aunque con diferencias internas de rendimiento y comandos que, para el día a día, apenas se notan. Merece la pena probarlo en algún momento.

4. Configurando module federation

4.1. Los parámetros clave del plugin

Toda la configuración de module federation vive dentro del `ModuleFederationPlugin` de webpack, y gira en torno a un puñado de propiedades:

- ▶ `name`: el nombre público con el que la aplicación se identifica.
- ▶ `remotes`: objeto que define los elementos que se van a obtener de manera remota, en tiempo de ejecución, de otros módulos federados. La clave es el nombre que usará la aplicación consumidora para referirse al MFE remoto, y el valor combina el nombre real del MFE remoto con la ruta a su archivo de entrada (`remoteEntry`). Por ejemplo: `"my-nav": "nav@http://localhost:3000/remoteEntry.js"`.
- ▶ `exposes`: objeto con lo que la aplicación expone y que otras pueden consumir mediante imports, una vez configurado el remote correspondiente. La clave es la ruta pública y el valor la ruta interna real, por ejemplo: `"/Header": "./src/Header"`.
- ▶ `fileName`: el nombre del archivo que gestiona las peticiones en tiempo de ejecución; por convención suele llamarse `remoteEntry.js`.

- ▶ **shared:** el listado de dependencias que la aplicación compartirá con otras para dar soporte a lo que expone en exposes. Su gran ventaja es que evita colisiones de versión: cada componente usa la que necesita y, si otro ya cargó la misma versión, no se vuelve a cargar.

4.2. Analizando un ejemplo completo

Un ejemplo con dos aplicaciones, Nav y Home, ayuda a ver cómo encajan estas piezas. Nav se configura como remote: define su `output.publicPath` en `http://localhost:3001/`, su `fileName` como `remoteEntry.js`, y expone su componente Header a través de exposes (`"./Header": "./src/Header"`). Home, por su parte, se configura como host: declara remotes con la entrada `"my-nav"`:

`"nav@http://localhost:3001/remoteEntry.js"`, y desde su código puede importar dinámicamente ese componente con algo como `const Header = React.lazy(() => import("my-nav/Header"))`.

Un segundo ejemplo, ya con más piezas en juego, muestra una aplicación llamada app que expone tres componentes (Button, Card y CardList), declara un remote llamado app2 apuntando a `http://localhost:3001/remoteEntry.js`, y comparte react y react-dom como singleton, tomando la versión exacta que indique su `package.json`. Para consumir cualquier elemento de app2 basta con importarlo como un módulo más: `import("app2/SomeElement")`.

Además de name, remotes, exposes, fileName y shared, la configuración necesita otros tres detalles:

- ▶ `output.publicPath`: la URL donde se va a levantar el microfrontal; es la misma ruta que otras aplicaciones usarán en su remotes.
- ▶ `resolve.extensions`: la lista de extensiones de archivo (`.tsx`, `.ts`, `.js`, `.jsx`, `.json...`) que webpack probará si la importación no especifica una.
- ▶ El propio plugin hay que requerirlo explícitamente: `const ModuleFederationPlugin = require("webpack/lib/container/ModuleFederationPlugin")`.

5. Dependencias compartidas

La ventaja más llamativa de module federation es compartir dependencias entre aplicaciones distintas de forma global, pero conviene entender bien su letra pequeña antes de darla por gratuita.

Las dependencias del proyecto se declaran, como siempre, en el `package.json` (webpack, babel, css-loader, sass y demás). Esas dependencias son las que viajan en el bundle final una vez pasado el proceso de tree shaking, que elimina el código que no se usa. Aquí está el matiz importante: ese tree shaking no se aplica de la misma forma a las dependencias que se comparten mediante module federation.

Si varios microfrontales comparten las mismas dependencias sin más, cada bundle termina cargando su propia copia completa, inflando su peso una y otra vez. Module federation resuelve justo esto: permite compartir esas dependencias comunes de forma global, reduciendo el peso de cada bundle individual y, con ello, el tiempo de la primera carga.

Para conseguirlo, la propiedad `shared` del `ModuleFederationPlugin` admite, para cada dependencia, dos parámetros:

- ▶ **singleton**: indica que debe existir una única copia de esa dependencia por aplicación. Es imprescindible para dependencias con estado (stateful) como React o Angular, donde tener dos copias distintas rompería la aplicación.
- ▶ **requiredVersion**: la versión que necesita la aplicación, que debería coincidir con la indicada en su package.json —de ahí que normalmente se importen directamente desde ahí— y que determina si se puede reutilizar una versión ya cargada por otro MFE o si hace falta cargar una nueva.

Aquí llega la pregunta clave: ¿por qué no compartir sencillamente todas las dependencias del package.json? La respuesta está en cómo se cargan: las dependencias compartidas por module federation se cargan en tiempo de ejecución, no en tiempo de compilación, así que hay que poner a disposición la dependencia completa por si alguien la necesita entera. Eso significa que no se les aplica tree shaking, y que el bundle resultante pesa más de lo que pesaría si esa dependencia estuviera empaquetada normalmente.

La recomendación práctica es doble: compartir únicamente las dependencias que de verdad son comunes entre aplicaciones, y procurar que estén lo más modularizadas posible para compartir solo la parte que realmente se necesita. El ejemplo clásico es una librería de calculadora: en lugar de compartirla entera, conviene que exponga por separado la suma, la resta, la multiplicación y la división (@miCalculadora/sum, @miCalculadora/sub...), cada una con su propio singleton y requiredVersion. Compartir cada paquete por separado es, en la práctica, una especie de tree shaking manual.

★ Lo más importante

- ▶ Las dependencias compartidas se cargan en tiempo de ejecución y no pasan por tree shaking: comparte solo lo estrictamente necesario.
- ▶ singleton evita que convivan varias copias de una dependencia con estado (React, Angular...); requiredVersion decide si se puede reutilizar una versión ya cargada.
- ▶ Cuanto más modularizada esté una dependencia compartida, más se parece su reparto a un tree shaking manual.

6. Cómo funciona module federation por dentro

En el proceso de compilación, el plugin de federación de módulos realiza, en esencia, tres tareas:

- ▶ Reúne todos los elementos que la aplicación va a exponer (definidos en exposes) y genera el remote entry file, que actúa como un mapa para que otras aplicaciones sepan qué pueden cargar y desde dónde.
- ▶ Gestiona lo declarado en remotes: qué módulos remotos necesita esta aplicación y dónde encontrarlos.
- ▶ Gestiona las dependencias declaradas en shared.

Para poder gestionar tanto los remotes como las dependencias compartidas, el remote entry file necesita actuar a la vez de mapa y de cargador. En tiempo de compilación se genera un bundle independiente para cada uno de estos elementos, de modo que el bundle principal de la aplicación no incluye todo lo que expone —solo lo esencial— y el resto queda disponible como bundles separados, listos para cargarse solo si hace falta.

Cuando la aplicación arranca, lo primero que se carga es precisamente el remote entry file. Este busca, primero, si alguna aplicación ya ha cargado una versión compatible de la dependencia compartida que

necesita; si es así, la reutiliza sin volver a descargarla. Si no la encuentra, carga el bundle correspondiente, generado ya en tiempo de compilación.

Todo esto es posible porque cada dependencia que se va cargando se registra en una variable global, que es precisamente la que el remote entry file consulta antes de decidir si carga algo nuevo o reutiliza lo que ya está disponible.

7. Qué hemos aprendido

Module federation cierra el círculo abierto en el documento anterior: una vez sabemos dividir el frontend en microfrontales bien delimitados por dominio, esta funcionalidad de Webpack 5 es la que permite compartir código entre ellos en tiempo de ejecución, sin recompilar nada y sin depender de un framework propio.

Los conceptos que conviene retener son los papeles de host y remote que un mismo MFE puede combinar, los parámetros del plugin —name, remotes, exposes, fileName y shared—, y la advertencia sobre las dependencias compartidas: al cargarse en tiempo de ejecución no pasan por tree shaking, así que hay que ser selectivo y modular con lo que se comparte.

No conviene perder de vista tampoco su otra cara: es una técnica compleja, pensada para casos donde de verdad hace falta actualizar código compartido sin recompilar; si ese no es el escenario, un paquete de NPM tradicional sigue siendo la opción más simple.